

COMPACT Data Pipeline Development

DZA

22 May, 2026

Table of Contents

1 Data Pipeline Overview: 4

Meerkat Pipeline Development Agenda:

2:00 – 2:15 pm, **Savin** - Meerkat data introduction.

2:15 – 2:35, **Torsten** - Polarimetric Imaging Reflecting Stokes Relations

2:35- 2:50, **Aritra** - Searching for axions at high time resolution using voltage dump

10 minutes break

3:00 – 3:15 (Not confirmed), **Michael** – Lausitz array data processing in the context of Meerkat

3:15 – 3:30, **Julian/Anton** - Bayesian Radio Correlator

3:30- 3:45, **Adrian** - Bayesian Radio Camera – Data processing

3:45 – 4:15: **Discussion**

PI	Project Name	Data product required	Time resolution, Frequency resolution	Compute requirements
Savin	AI based source detection	High dynamic range images	2 hour integration images	
Torsten	Polarimetric imaging	Visibilities	long hour integration images	
Aritra	Search for axions	Visibilities	millisecond scales	
Julian/ Anton	Radio Correlator	Visibilities needed	millisecond time scales	
Adrian	Radio Camera	Voltage dumps	voltage data	

1 Data Pipeline Overview:

1. Overall Pipeline design:



2. Overview diagram from the airflow UI:



3. Airflow Setup:

- a. Here we will use the docker compose version which is far easier than the local installation setup. Airflow is notorious for dependencies.
- b. Download the docker compose file and follow the instruction listed at <https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html#fetching-docker-compose-yaml>
- c. Creates a different ssh key private and public key without passphrase (named as airflow_key). Not the best solution, but there is a work around but need to look deeper.
- d. update the docker compose mounting volumes as follows: Also save the AIRFLOW_UID=1001 into a .venv file on the same docker-compose.yaml location. The ssh keys created on your local machine will be mounted on to the docker for ssh ing into the remote machines.

- e.


```
volumes:
  - ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
  - ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
  - ${AIRFLOW_PROJ_DIR:-.}/config:/opt/airflow/config
  - ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
  - ${AIRFLOW_PROJ_DIR:-.}/data_files:/opt/airflow/data_files
  - ~/.ssh:/home/airflow/.ssh:ro
user: "${AIRFLOW_UID:-50000}:0"
```

- f. Now start running all the docker services.

- g.


```
docker compose up -d
```

Connect to the api server at <http://localhost:8080>. Login to the user Interface and set up connections as follows. For the default user, username: airflow, password: airflow. Similarly create a connection for the GPU machine as well.

h.

- i. Now add your files to the dag folder and if any additional data/folders are required, just mount it using the same docker compose yaml file.
 - j. Now do : git clone <https://gitlab.dzastro.de/dza/data-science/std-img-pipeline.git> In the cloned repo: "dags, data_folders" are needed in your local directory containing the docker compose yaml file. "scripts and util_scripts" needs to be on the login node of your HPC account. Also a shared workspace needs to be created which can be accessed from the barnard and capella clusters.
 - k. Once everything is set up, trigger the test dags from the UI and check the logs to see if everything is working.
4. Pipeline setup:
- a. Create a workspace in horse/walrus/cat filesystem. Create directories named as "prod" and inside "mkdir -p ./batch_logs ./images ./uvh5 ./ms ./stimela_logs ./stimela_run_dir ./txt_files ./yaml "
 - b. Update the file paths of the batch and bash scripts in the HPC and their corresponding names in the dag file locally.
 - c. For the correlation, the standard correlator python code will be used. Alternatively a singularity container can be used as well.
 - d. Stimela part requires a full operational stimela pipeline installed and tested using SLURM.
 - e. yq (Go based) needs to be installed for the editing existing yaml and creating new yaml files.